

Lecture 2

Debugging and Unit Testing

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem
Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Ideal Properties of Long-Lived Programs

Non-throwaway code is ideally:

- Free of bugs
- Easy to understand
- Easy to change

Q1: How do we systematically debug programs?

Q2: How do we build confidence in the correctness of our code?

Example

- Does this function always do what it is intended to do?

```
def count_vowels(s: str) -> int:  
    return sum(s.count(ch) for ch in 'aeiou')
```

- Note: Input is expected to be comprised of only ASCII values
 - No è, ę, æ, etc.

How do we systematically verify this?

Equivalence Partitioning

Equivalence partition: Set of values that behave the same way with respect to what is being tested

- *Input equivalence partition:* Set of values producing same “kind” of output
- *Output equivalence partition:* Set of unique “kinds” of outputs

Equivalence Partitioning, continued

- Example:

```
def my_abs(n: int) -> int:  
    return n if n >= 0 else -n
```

- Equivalence partitions:
 - $n \geq 0$ (output is n for all of them)
 - $n < 0$ (output is $-n$ for all of them)

Try to have test cases that cover all equivalence partitions (or combinations thereof)

Types and Equivalence Partitions

Values of data types may be grouped into partitions

Data type	Possible partitions
int	Positive, zero, negative, very small/large integers
str	Empty string, uppercase, lowercase, spaces, symbols, <i>(and combinations thereof);</i> non-English characters, emoji

Data type	Possible partitions
<code>list[...]</code> , <code>set[...]</code> , <code>dict[...]</code>	Empty, singleton, duplicates, some equal, all equal, all unique

Types and Equivalence Partitions

The bug in `count_vowels` should be easier to identify through this lens

- Equivalence of values in partition still depends on behavior of what is being tested

Practice: Close Call

Problem Statement

Mario, Wigi, and Pearl are racing to be the one to get the star!

Mario, Wigi, Pearl, and the star are all on the number line. Mario is located at position m , Wigi at position w , and Pearl at position p . No two of them are at the same location.

The star is at position s . No one is at the star's location yet.

If everyone runs at the same rate toward the star, who gets to the star first? If two people get to the star first at the same time, the star disappears—no one gets the star.

Create a function `who_gets_the_star` that takes in four integers m , w , p and s , and returns one of the strings:

- `MARIO` (if Mario gets the star)
- `WIGI` (if Wigi gets the star)
- `PEARL` (if Pearl gets the star)
- `NONE` (if no one gets the star)

Sample Calls

```
who_gets_the_star(4, 9, 1, 7) == "WIGI"  
who_gets_the_star(5, 9, 1, 7) == "NONE"
```

Copy

Limits

- $1 \leq m, w, p, s \leq 100$

Practice: Close Call

Q1 + Q2: For each attempt below, give some (m, w, p, s) it gets wrong and explain why it does:

Practice: Close Call

```
# Q1
def who_gets_the_star(m: int, w: int,
                      p: int, s: int):
    scoreboard = {
        "MARIO": abs(s - m),
        "WIGI": abs(s - w),
        "PEARL": abs(s - p),
    }

    distances = list(scoreboard.values())
    removed_duplicates = set(distances)

    if len(removed_duplicates) != len(distances):
        return "NONE"

    winner = distances.index(min(distances))
    names = list(scoreboard.keys())

    return names[winner]
```

Practice: Close Call

```
# Q2
def who_gets_the_star(m: int, w: int,
                      p: int, s: int):

    test_m = s - m
    test_w = s - w
    test_p = s - p

    if test_m < test_w and test_m < test_p:
        return "MARIO"
    elif test_w < test_m and test_w < test_p:
        return "WIGI"
    elif test_p < test_w and test_p < test_w:
        return "PEACH"
    else:
        return "NONE"
```

Edge cases

Edge case: Unusual input that tests limits or special behavior of what is being tested

Boundary value: Edge case within or immediately around limits of input range

Edge cases

- Example:

```
def remove_nth_qualifier(qualifiers: list[Qualifier], n: int):  
    """Removes the nth element in the given list of  
    qualifiers."""  
    qualifiers.pop(n)
```

When would the above function fail to do what it is intended to do?

Ideal Properties of Long-Lived Programs

Non-throwaway code is ideally:

- *Free of bugs*
- Easy to understand
- Easy to change

Q1: How do we systematically debug programs?

Q2: How do we build confidence in the correctness of our code?

Motivating Example

- Python script that merges CSV files, but ignores rows with already seen `id` value:

Motivating Example

```
import csv
import glob

all_rows: list[dict[str, str]] = []

for filepath in sorted(glob.glob(f"{input_dir}/*.csv")):
    with open(filepath) as f:
        reader = csv.DictReader(f)
        all_rows.extend(reader)

seen_ids: set[str] = set()
for row in all_rows:
    if row["id"] in seen_ids:
        all_rows.remove(row)
    else:
        seen_ids.add(row["id"])

with open(output_file, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=all_rows[0].keys())
    writer.writeheader()
    writer.writerows(all_rows)
```

Motivating Example

This contains a bug; how do we identify and fix it?

- Assume all files are valid CSVs that have `id` and consistent headers

Search Space Reduction

```
import csv
import glob

# all_rows: list[dict[str,str]] = []
#
# for filepath in sorted(glob.glob(f"{input_dir}/*.csv")):
#     with open(filepath) as f:
#         reader = csv.DictReader(f)
#         all_rows.extend(reader)

all_rows = [
    [...],
    [...],
    [...],
]
```

Search Space Reduction

```
seen_ids: set[str] = set()
for row in all_rows:
    if row["id"] in seen_ids:
        all_rows.remove(row)
    else:
        seen_ids.add(row["id"])

with open(output_file, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=all_rows[0].keys())
    writer.writeheader()
    writer.writerows(all_rows)
```

Narrows location of bug in code down

- Split code into independent units

Unit Testing

Unit testing: Automated testing of program units

- Goal: Show that individual units of a program behave correctly
 - “Units” are usually functions or objects; isolated testing of functions/objects
- Builds confidence that extensions to code do not break existing functionality
- Many other kinds of tests (*will not be covered in CS 12*)
 - *Stress testing*
 - *Fuzz testing*

Pytest

- One of several test runners for Python (installed separately)
- Runs all functions named `test_*` in all `test_*.py` in current working directory:

Pytest

```
from main import count_vowels    # Change `main` to `<filename>`

# Note: Not enough!
def test_count_vowels():
    assert count_vowels("Hello, world!") == 3    # Note: Output is
    shown even if
    assert count_vowels("aeiouaeiou") == 10      # no error message is
    given
    assert count_vowels("Nymph") == 0
```

- Run via `pytest` or `python -m pytest`

Test Coverage

Test/Code coverage: Percentage of parts of code executed by test suite

- *Test suite*: Collection of unit tests
- *Line coverage*: Test coverage of lines in source/assembly (not covered)
- *Branch coverage*: Test coverage of true/false conditionals in source/assembly (not covered)

Test Coverage

“Program testing can be used to show the presence of bugs, but never to show their absence!”

— E.W. Dijkstra (*from Notes On Structured Programming*)

Test Coverage

- 100% coverage does **not** prove that the program has no bugs
- For larger programs: 80%-90% coverage is good enough (diminishing returns)
 - High degree of confidence that code works as intended
- `coverage run -m pytest`: Produces test coverage information
 - `coverage html` for HTML output
 - May need to run `pip3 install pytest-cov`

Back to Motivating Example

- Refactored into smaller units (functions):

```
import csv
import glob

def read_csvs(input_dir: str):
    rows: list[dict[str, str]] = []

    for filepath in sorted(glob.glob(f"{input_dir}/*.csv")):
        with open(filepath) as f:
            reader = csv.DictReader(f)
            rows.extend(reader)

    return rows
```

Back to Motivating Example

- Refactored into smaller units (functions):

```
def remove_duplicates(rows: dict[str, str]):
    seen_ids: set[str] = set()
    for row in rows:
        if row["id"] in seen_ids:
            rows.remove(row)
        else:
            seen_ids.add(row["id"])

def write_to_file(output_file: str, rows: dict[str, str]):
    with open(output_file, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=rows[0].keys())
        writer.writeheader()
        writer.writerows(rows)
```

Summary

Make code easier to debug by decomposing it into units

Identify a test case that reproduces the bug

Consider equivalence partitions and edge cases when making test cases

Test units in isolation to narrow location of bug down

Have unit test suite with high code coverage